

KOTLIN

* Syntax:

```
fun main() {
```

```
    ...
```

```
}
```

- print() - print without space
- println() - print with space.

synt: print(---)
println(---)

* Variable: ~~Int, Float, Double, String~~

- Integers: ~~int~~, byte, short, long
- Unsigned integers: UByte, Ushort, UInt, Ulong
- Floating point: float, Double
- Booleans: Boolean
- Character: char
- String: String

Syntax: ~~var~~/val var variable: type = data

var - Dynamic Data

val - Static Data

* Collection:

- List: Ordered Collections of items
- Sets: ~~Unordered~~ unordered collection of items.
- ~~Map~~ Maps: Sets of Keys.

* List: stores items in the order that they are added, and allows duplicate items.

Syntax: var variable = listOf (data1, data2, data¹, ..., dataⁿ)

• ~~first~~ variable.first() = first of list

• last() = last of list

• ~~count()~~

• count() = no of elements in list

in operator = Check an item in list.

• add() ("data") = to add

• remove("data") = to remove

* Set: unordered and only unique items.

Syntax: var variable = setOf (d1, d2, d³, ..., dⁿ)

same as above.

* Map: stores item as key value pair.

Syntax: var variable = mapOf (d1 to N1, d2 to N2, ..., dⁿ to Nⁿ)
 print (~~read~~ variable [d1]) → Output N1

Variable ["d4"] = N4 - adds item to map.
 only for mutable map

Variable . remove (d1) - remove ~~var~~

• count()

• containsKey (d1) - checks if d1 is in map

- Keys → prints Keys
- Values → prints values

in

~~var~~ ^{var} . Uppercase () → ~~make~~

var . lowercase () →

var . starts with (char) →

* if :

Synt !

if (condition) {

 ;

else {

 ;

if can be assign
value to variables

* When :

~~When (cond) {~~

~~;~~

OR

When (Obj) {

cond1 → statement 1

cond2 → statement 2

⋮

cond n → statement n

else → statement.

;

it is

When can assign
value to variables.

(OR)

when {

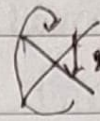
condition 1 →

condition 2 →

condition 3 →

else →

}



... - to

Eg 1..3 op 1 2 3

... < - excluding the it Eg 1.. < 3 op 1 2

downto - opposite of ..

loops-for:

for (var in range) { }

While:

while (var < range) { }

}

do-while:

do { }

{ while (var < range)

PFO

* Userdefined function:

```
fun funname () { (var : type) {
    operation
}
```

```
or fun main () {
    funname (data)
}
```

* lambda Expression:

① creating fun in main itself -
eg:

```
main () { fun main () {
```

```
    val uppercase : String
    val uppercaseString = { text : String → text.toUpperCase() }
    println ("uppercaseString ("hello"))
}
```

2-

* Pass to another functionVariable type Conversion:

.toType() → Replace type with type.

or asType. ~~not~~ as?Type

- is formatted but used for readability.

and - AND

and, or, xor → Basic Bits operation
 shl → shift left Replace right most with 0
 shr → shift right, replace leftmost with 0.

BigInteger & BigDecimal → Infinite Size,
 but have to import java.math.BigDecimal
 import java.math.BigInteger

Character.code = gives ASCII value.

String.trimIndent() = ^{used for multiline.} removes unnecessary space. " " "

"\n" or any other special.

list var. join to string() → makes one string.

• joinToString(separator = " ") → use it between elements of list (default = ,)
 • You can just write (-)

• joinToString(limit = n) → will display first then and rest will be ...

* User defined function with arg:

fun name (variable : type = data / arg) declared.

✓ this will be used if not declared.

* Function Overloading: declaring function with same name in same file as long as the parameter is different.

- equals () : compares instances of class
- copy () : copies the class.
- break - terminate
- Continue - skip and move to next epoch.
- Size () = size of list
- indices () = available indices.

* Nullability:

1. Every property needs to have an explicit value. There is no such thing as an implicit null value.
2. A regular type does not accept the null value.
3. To specify a nullable type, you need to end a regular type with a question mark (?).
4. Nullable value cannot be used directly. They must be used safely or cast first (using one of the tools presented later)

* Safe Calls: using (?.) instead of (.). Used for those who return null.

* Elvis Operator: (?:)